TrustRAG

Complete Project Teaching Guide

A Trustworthy Offline Retrieval-Augmented Generation Framework with Explainable Multi-Signal Hallucination Detection

---

Student: Archa Biju (CB.SC.P2CSE25001)
Guide: Dr. Ramya G R
Institution: Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore
Programme: M.Tech Computer Science — Review 2

This guide teaches the project end-to-end: the research novelty, the literature it builds on and all 30 references it cites, the system architecture and every mathematical formulation behind it, the tools chosen and why, a file-by-file walkthrough of the actual code, and a candid assessment of exactly how much is implemented and verified today versus what the paper describes as the target experimental design.

Table of Contents

- 1. Project Overview & Problem Statement
- 2. Novelty & Contributions
- 3. Literature Positioning & Full Reference List (30 references)
- 4. System Architecture
- 5. Algorithms & Mathematical Formulation
- 6. Tools & Technology Stack — Why Each Was Chosen
- 7. Code Implementation Walkthrough (file by file)
- 8. Implementation Status — What's Verified vs. What's Claimed
- 9. Datasets — Current State & Recommended Expansion
- 10. Roadmap for Semester 3

1. Project Overview & Problem Statement

Large Language Models (LLMs) write fluently, but fluency is not the same as truth. An LLM predicts the next most probable token, not the factual correctness of what it says, so it can produce confident, well-formed sentences that are simply wrong — a failure mode called **hallucination**. This is a serious problem the moment an LLM is used for anything where being wrong has real consequences: healthcare, law, enterprise knowledge management, defense, or any setting where a user has no independent way to check the answer.

Retrieval-Augmented Generation (RAG) is the standard fix: instead of trusting only what the model memorized during training, first retrieve relevant real documents, then give those documents to the model as context before it answers. This grounds the response in actual evidence rather than the model's parametric memory. But RAG alone does not solve the problem — a language model can still ignore the retrieved evidence, blend it with invented details, or misread it, producing a sentence that *sounds* grounded but is not actually supported by anything that was retrieved. This residual failure mode is sometimes called **RAG hallucination**, and it is the exact problem this project addresses.

The project, TrustRAG, builds a complete offline RAG pipeline and adds a verification layer after generation: every sentence the model produces is checked against the evidence that was actually retrieved for it, using three independent signals (logical entailment, semantic similarity, and embedding alignment), and the system decides — sentence by sentence and then response by response — whether to show the answer with citations, show it with a warning, or block it outright and hand back the raw evidence instead.

Crucially, the entire system runs **offline**: the embedding model, the vector index, the generator LLM, and the verification models all run locally with no cloud API and no data leaving the machine, which is what makes it viable for privacy-sensitive deployments (hospitals, defense, enterprise document stores) where sending documents to a third-party API is not an option.

Research questions this project answers:

- Can a locally-run RAG pipeline verify its own generated answers against retrieved evidence, without calling any external fact-checking API?
- Does combining three independent, complementary signals (NLI entailment, BERTScore, cosine similarity) catch more hallucinations than any single signal alone?
- Can the verification signals themselves double as an explainability layer — showing *why* an answer was trusted or blocked — rather than requiring a separate, bolted-on interpretability tool?

2. Novelty & Contributions

The paper's own framing of the research gap is precise and worth stating exactly: prior work has studied retrieval quality, hallucination detection, factual-consistency checking, and explainability **as separate problems**. Very few approaches address all four together in one framework, and almost all rely on cloud infrastructure, which rules them out for offline, privacy-sensitive deployment. TrustRAG's novelty is combining all four into a single, offline, unified pipeline.

The five contributions claimed in the paper, stated in detail:

- **1. A unified trustworthy offline RAG system.** TrustRAG combines semantic retrieval, explainable evidence attribution, and multi-signal hallucination detection inside one response-validation pipeline, rather than as separate tools bolted together after the fact.
- **2. The Multi-Signal Trust Score.** A single reliability number per sentence, computed as a weighted combination of NLI entailment, BERTScore, and cosine similarity — three signals that each fail differently, so combining them catches more than any one alone (verified in the paper's own ablation study, Section 5).
- **3. An intrinsic explainability module.** Provenance tracking and evidence attribution are not a post-hoc explanation layer added after the answer is generated — they are computed as part of the same retrieval and verification pipeline, so every trust decision already comes with its own justification.
- **4. A trust-aware tri-state decision system.** Rather than a binary correct/incorrect label, each sentence is classified Supported, Weakly Supported, or Hallucinated, giving both sentence-level nuance and a response-level safety control (suppressing the whole answer when too much of it is unsupported).
- **5. Demonstrated superiority over a conventional RAG baseline** on hallucination rate, provenance, and semantic alignment (see Section 8 of this guide for what has actually been re-verified on the current codebase versus what is reported in the paper draft).

2.1 How TrustRAG compares to prior approaches (paper's Table 1)

The paper positions TrustRAG against representative prior approaches on five capabilities. This comparison is the clearest one-page argument for the novelty claim:

Approach	Dense Retrieval	Explainable Attribution	Multi-Signal Detection	Offline / Local	Trust Score
Lewis et al. (RAG)	Yes	No	No	No	No
REALM	Yes	No	No	No	No
Dense Passage Retrieval (DPR)	Yes	No	No	No	No
LIME / SHAP	No	Partial	No	No	No
SelfCheckGPT	No	No	Yes	Yes	No
FactScore	No	No	Yes	Yes	No
RAGAS	Yes	Partial	Yes	Yes	No
RLSeek	Yes	Partial	Yes	No	No
Evidence-Aligned Entity Verification	Yes	Partial	Yes	No	No
MARCH	Yes	No	Yes	No	No
TrustRAG (this project)	Yes	Yes	Yes	Yes	Yes

Every row except the last comes directly from the paper's literature review (Section II) and its comparison table. This table is the paper's own argument, reproduced faithfully — it is not a result this guide is independently verifying.

3. Literature Positioning & Full Reference List

3.1 How the literature review is structured

The paper's related-work section (Section II) is organized into three streams that TrustRAG then unifies:

- **(A) LLMs and Retrieval-Augmented Generation** — from the Transformer architecture and BERT through RAG, REALM, and Dense Passage Retrieval, establishing that retrieval improves grounding but cannot fully eliminate hallucination on its own.
- **(B) Hallucination detection in LLMs** — FactCC and SummaC (early NLI-based factual-consistency checking), SelfCheckGPT and FActScore (zero-resource and atomic-fact verification), RAGAS (automated RAG evaluation), and a cluster of very recent 2026 work (RLSeek, Evidence-Aligned Entity Verification, Logic Matters, MARCH, Faithfulness-Aware Uncertainty Quantification) — all of which treat detection as a separate post-hoc evaluation step rather than an integrated part of the generation pipeline.
- **(C) Explainable AI for trustworthy LLMs** — LIME and SHAP as the standard model-agnostic explanation tools, and the observation that neither was designed to explain a multi-component RAG pipeline (retrieval plus generation plus verification), which is exactly the gap TrustRAG's intrinsic explainability module fills.

3.2 Full reference list (30 references, as cited in the paper)

These are the exact 30 references from the paper's bibliography, reproduced verbatim so they can be reused directly in the thesis or any derived document.

[1] T. B. Brown et al., “Language Models are Few-Shot Learners,” Adv. Neural Inf. Process. Syst., vol. 33, pp. 1877–1901, 2020.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of Hallucination in Natural Language Generation,” ACM Comput. Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[3] L. Huang et al., “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” ACM Trans. Inf. Syst., vol. 43, no. 2, pp. 1–55, 2025.

[4] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Adv. Neural Inf. Process. Syst., vol. 33, pp. 9459–9474, 2020.

[5] K. Guu, K. Lee, Z. Zhong, P. Pasupat, and M. Chang, “REALM: Retrieval-Augmented Language Model Pre-training,” Proc. Int. Conf. Mach. Learn. (ICML), pp. 3929–3938, 2020.

[6] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” Proc. Conf. Empirical Methods Natural Language Processing (EMNLP), pp. 6769–6781, 2020.

[7] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval Augmentation Reduces Hallucination in Conversation,” Findings Assoc. Comput. Linguistics: EMNLP, pp. 3784–3803, 2021.

[8] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,” Proc. Conf. Empirical Methods Natural Language Processing (EMNLP), pp. 9004–9017, 2023.

[9] S. Min et al., “FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long-Form Text Generation,” Proc. Conf. Empirical Methods Natural Language Processing (EMNLP), pp. 12076–12100, 2023.

[10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval-Augmented Generation,” Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics: Syst. Demonstrations, pp. 150–158, 2024.

[11] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You?: Explaining the Predictions of Any Classifier,” Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD), pp. 1135–1144, 2016.

[12] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Adv. Neural Inf. Process. Syst., vol. 30, 2017.

[13] A. Williams, N. Nangia, and S. Bowman, “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT), pp. 1112–1122, 2018.

[14] W. Kryscinski, B. McCann, C. Xiong, and R. Socher, “Evaluating the Factual Consistency of Abstractive Text Summarization,” Proc. Conf. Empirical Methods Natural Language Processing (EMNLP), pp. 9332–9346, 2020.

[15] P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst, “SummaC: Revisiting NLI-Based Models for Inconsistency Detection in Summarization,” Trans. Assoc. Comput. Linguistics, vol. 10, pp. 163–177, 2022.

[16] O. Honovich et al., “TRUE: Re-evaluating Factual Consistency Evaluation,” Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT), pp. 3905–3920, 2022.

[17] P. He, J. Gao, and W. Chen, “DeBERTa-v3: Improving DeBERTa Using ELECTRA-Style Pre-training with Gradient-Disentangled Embedding Sharing,” Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[18] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” Proc. Int. Conf. Learn. Represent. (ICLR), 2020.

[19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT), pp. 4171–4186, 2019.

[20] A. Vaswani et al., “Attention Is All You Need,” Adv. Neural Inf. Process. Syst., vol. 30, 2017.

[21] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” Proc. Conf. Empirical Methods Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992, 2019.

[22] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[23] A. Grattafiori et al., “The Llama 3 Herd of Models,” CoRR, vol. abs/2407.21783, 2024.

[24] Z. Huang et al., “RLSeek: Evidence-Grounded Reasoning for RAG Hallucination Detection,” Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL), pp. 32329–32347, 2026.

[25] R. Jia et al., “Evidence-Aligned Entity Verification for Hallucination Detection in Retrieval-Augmented Generation,” Findings Assoc. Comput. Linguistics (ACL), pp. 29538–29554, 2026.

[26] W. Chen et al., “Rethinking Evaluation for LLM Hallucination Detection: A Desiderata, A New RAG-Based Benchmark, and New Insights,” Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL), pp. 14912–14931, 2026.

[27] N. Yang and K. Huang, “Logic Matters in Lightweight Hallucination Classification for RAG Systems,” Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL), pp. 1605–1617, 2026.

[28] Z. Li et al., “MARCH: Multi-Agent Reinforced Check for Hallucination,” Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL), pp. 39389–39415, 2026.

[29] E. Fadeeva et al., “Faithfulness-Aware Uncertainty Quantification for Fact-Checking the Output of Retrieval-Augmented Generation,” Findings Assoc. Comput. Linguistics (ACL), 2026.

[30] L. M. Amugongo, P. Mascheroni, S. Brooks, S. Doering, and J. Seidel, “Retrieval Augmented Generation for Large Language Models in Healthcare: A Systematic Review,” PLOS Digit. Health, vol. 4, no. 6, Art. no. e0000877, Jun. 2025.

4. System Architecture

TrustRAG is a six-stage pipeline. A query enters at the top, and a verified, citation-carrying (or blocked) response comes out the bottom. Each stage below is implemented by a specific module in `src/`, named so in parentheses.

Stage	What it does	Module
1. Ingestion	Load PDFs/DOCX/TXT/MD from data/corpus/ and split into overlapping 512-word chunks (50-word overlap)	src/ingestion/loader.py src/ingestion/chunker.py
2. Retrieval	Embed the query and all chunks with Sentence-BERT (all-MiniLM-L6-v2, 384-dim), index chunks in FAISS IndexFlatIP, and return	src/retrieval/embedder.py src/retrieval/faiss_store.py
3. Generation	Send the query plus retrieved chunks to a local LLM (Mistral/LLaMA/Phi-3 via Ollama's HTTP API) with a system prompt instructing	src/generation/llm_client.py
4. Explainability (XAI)	Split the generated answer into sentences; for each sentence, find its best-matching source chunk and compute a provenance score (f	src/explainability/xai_layer.py
5. Multi-signal detection	Score each sentence with three independent signals: NLI entailment (DeBERTa cross-encoder), BERTScore F1 (DistilBERT token	src/detection/nli_detector.py src/detection/bertscore_detector.py src/detection/hallucination_engine.py
6. Decision control	Classify each sentence Supported / Weakly Supported / Hallucinated against configurable thresholds, and mark the whole response Ha	src/control/decision_controller.py

All six stages are orchestrated by `src/pipeline.py`'s `RAGHallucinationPipeline` class, and every weight/threshold above is read from `config/config.yaml` rather than hard-coded — so the whole pipeline can be re-tuned without touching source code. The Streamlit app in `ui/app.py` is the dashboard on top of this pipeline.

5. Algorithms & Mathematical Formulation

This section reproduces the paper's formal notation for each stage, and immediately after each block, states exactly which function in the real codebase implements it — so the math and the code are taught together.

5.1 Dense Semantic Retrieval

Query q = {w_1, ..., w_n}; knowledge repository D = {d_1, ..., d_n}. Both are embedded with Sentence-BERT:

```
e_q = f_SBERT(q)    e_di = f_SBERT(d_i)
```

Similarity is cosine similarity between normalized vectors (equivalent to a dot product):

```
S(q, d_i) = (e_q · e_di) / (||e_q|| · ||e_di||)
```

The top-k most similar chunks are retrieved: R = TopK(S(q, d_i)).

Code: `Embedder.embed()` in embedder.py calls `SentenceTransformer.encode(..., normalize_embeddings=True)` — normalizing at embedding time is exactly what makes a plain dot product equal cosine similarity later. `FAISSStore.search()` in faiss_store.py then runs the top-k search on an `IndexFlatIP` index, which computes that same inner product against every stored chunk vector and returns the k largest.

5.2 Explainable Evidence Attribution

The LLM generates G = f_LLM(q, R), split into sentences G = {g_1, ..., g_m}. For sentence g_j and retrieved document r_i, attribution is again cosine similarity between their embeddings:

```
A_ij = (e_gj · e_ri) / (||e_gj|| · ||e_ri||)
```

Documents are ranked per sentence: R*_j = Sort(A_1j, …, A_kj).

Code: `compute_chunk_attribution()` in xai_layer.py computes exactly this sentence–chunk similarity matrix and keeps the best-matching chunk per sentence as `attribution_score`, with a fixed acceptance threshold of 0.30 (`attributed = best_score >= 0.30`). `compute_provenance_score()` then reports what fraction of sentences cleared that bar.

5.3 Multi-Signal Hallucination Detection

Each sentence g_j is scored by three independent, complementary signals against the retrieved evidence R:

```
S_NLI,j = f_NLI(R, g_j)   (entailment probability, [0,1])
S_BERT,j = BERTScore(R, g_j)   (token-level F1, [0,1])
S_COS,j = (e_gj · e_R) / (||e_gj|| · ||e_R||),   e_R = (1/k)Σe_ri
```

Code: `NLIDetector.score_pair()` in nli_detector.py runs a `cross-encoder/nli-deberta-v3-small` CrossEncoder over (premise=chunk, hypothesis=sentence) and takes the entailment-class probability as S_NLI. `BERTScoreDetector.score_sentence()` in bertscore_detector.py computes token-level BERTScore F1 using a lightweight `distilbert-base-uncased` backbone and keeps the best-matching reference chunk's F1 as S_BERT. Cosine similarity S_COS reuses the same Sentence-BERT vectors from the retrieval stage. Each detector also ships a heuristic (lexical-overlap) fallback if its model fails to load, so the pipeline degrades gracefully rather than crashing.

5.4 Trust Score Computation

The three normalized signals are combined with fixed, config-driven weights:

$$T_j = w_1 \cdot S_{NLI,j} + w_2 \cdot S_{BERT,j} + w_3 \cdot S_{COS,j}, \quad \text{subject to } w_1+w_2+w_3 = 1,\ w_i \geq 0$$

The codebase's default weights (config/config.yaml, section `detection`) are **w_1 = 0.45 (NLI), w_2 = 0.35 (BERTScore), w_3 = 0.20 (Cosine)** — NLI is weighted highest because logical entailment is the strongest individual signal (confirmed by the paper's own ablation study, Table 4).

Code: `HallucinationEngine.compute_support_score()` in hallucination_engine.py implements this exact weighted sum, with an assertion that the three weights sum to 1.0 at construction time.

5.5 Tri-State Decision Controller

Two thresholds convert the continuous Trust Score into an interpretable state:

$$\tau_H = 0.75 \text{ (high)} \qquad \tau_L = 0.50 \text{ (medium)}$$

$$D(g_j) = \begin{cases} \text{Supported} & \text{if } T_j \geq \tau_H \\ \text{Weakly Supported} & \text{if } \tau_L \leq T_j < \tau_H \\ \text{Hallucinated} & \text{if } T_j < \tau_L \end{cases}$$

At the response level, the hallucination ratio HR = N_hall / N_sent is computed. If HR > 0.50, the entire response is suppressed and the raw verified evidence is returned instead of a partial, potentially misleading answer.

Code: `DecisionController` in decision_controller.py implements exactly this: `STATE_ANSWER` / `STATE_PARTIAL` / `STATE_BLOCK` map directly onto the paper's three states plus the response-level Blocked action, with `max_block_ratio = 0.5` read from config. Both this threshold logic and the 0.75/0.50 thresholds are exposed as live sliders in the Streamlit sidebar, so a reviewer can watch the classification change in real time.

5.6 Computational Complexity

Document embedding is a one-time offline cost, O(N·C_SBERT). Per query: encoding the query is O(C_SBERT); exact FAISS search over N vectors is O(N·d); evidence attribution across m sentences and k chunks is O(m·k·d); NLI and BERTScore inference are O(m·(C_NLI + C_BERTScore)) (model-dependent, the dominant cost in practice); and Trust Score plus decision are O(1) per sentence. Because retrieval restricts evidence to the top-k chunks before verification, the expensive model-dependent stage only ever runs on a bounded, small evidence set per sentence rather than the whole corpus.

6. Tools & Technology Stack — Why Each Was Chosen

Layer	Tool	Why this, specifically
Embedding	Sentence-BERT (all-MiniLM-L6-v2)	80MB, 384-dim, runs on CPU in milliseconds — no GPU or cloud API needed. Distilled Siamese-BERT train
Vector index	FAISS (IndexFlatIP)	In-memory, no server process, no network call. IndexFlatIP gives exact (not approximate) nearest-neighbor
Generation	Local LLM via Ollama (Mistral / LLaMA 3 / Phi-3)	Ollama packages open-weight models with one HTTP endpoint and handles quantization automatically, so a
NLI entailment	cross-encoder/nli-deberta-v3-small	A cross-encoder (both texts seen together in one forward pass) gives materially better entailment accuracy
Semantic similarity	BERTScore (distilbert-base-uncased backbone)	DistilBERT is roughly 40% smaller and faster than full BERT while retaining most of its representational qua
UI	Streamlit + Plotly	Lets a data pipeline become an interactive, chat-style dashboard (sliders, charts, expandable panels) in pur
Orchestration	Plain Python + PyYAML config	No heavyweight agent framework (e.g. LangChain) is used for the core pipeline; every stage is a small, inde

Why NOT a cloud LLM (GPT-4, Claude, Gemini)? Because the entire premise of the project is offline, privacy-preserving deployment — sending retrieved documents to a third-party API for generation would defeat the stated purpose, regardless of how much better a frontier model's fluency might be.

7. Code Implementation Walkthrough (file by file)

This section walks every core source file, what it actually does, and its key function signatures — read directly from the current codebase, not from a plan or a description of intent.

7.1 Ingestion layer

src/ingestion/document_loader.py

Loads raw PDF, DOCX, TXT, and MD files from data/corpus/ (extensions controlled by config.yaml's supported_extensions list) and returns a flat list of page dicts with source filename and page number attached, ready for chunking.

src/ingestion/chunker.py

Splits loaded text into overlapping windows. `chunk_text()` tokenizes with a simple whitespace split (no external tokenizer dependency), slides a window of chunk_size words with step = chunk_size – chunk_overlap, and tags every chunk with a UUID, source file, page number, and word offsets. Default config: 512 words per chunk, 50-word overlap. `get_sentences()` is reused later by the detection layer to split a generated answer into individually-verifiable claims.

7.2 Retrieval layer

src/retrieval/embedder.py

Wraps sentence-transformers. `Embedder.embed()` calls `SentenceTransformer.encode(..., normalize_embeddings=True)` — this single argument is what makes every later cosine-similarity computation in the pipeline reduce to a plain dot product. `embed_chunks()` batches the whole corpus at indexing time; `embed_query()` embeds one query at query time.

src/retrieval/faiss_store.py

Wraps FAISS. `build()` creates an `IndexFlatIP` and adds all chunk embeddings. `search()` returns the top-k chunks with a clamped [0,1] similarity_score and 1-indexed rank. `save()`/`load()` persist the index (faiss.index) and the parallel chunk metadata list (chunks_metadata.pkl via pickle) so re-embedding isn't needed on every app restart.

7.3 Generation layer

src/generation/llm_generator.py

`LLMGenerator.__init__()` pings Ollama's `/api/tags` endpoint to check connectivity and warns if the configured model isn't pulled yet. `build_prompt()` constructs a strict system prompt: answer only from the given context, and say so explicitly if the context is insufficient — this is the project's first line of defense against hallucination, before any verification happens. `generate()` POSTs to `/api/generate` with a 120-second timeout. If Ollama is unreachable or times out, `_fallback_response()` returns a clearly labeled '[FALLBACK – Ollama not running]' extractive answer built from the top retrieved chunk, so the rest of the pipeline (explainability, detection) can still be exercised and demoed even without a running LLM.

7.4 Explainability layer

src/explainability/xai_layer.py

The project's stated core novelty. Five functions: `compute_similarity_scores()` (query-to-chunk cosine, vectorized as a single matrix multiply); `compute_chunk_attribution()` (best-matching chunk per generated sentence, threshold 0.30); `compute_provenance_score()` (fraction of sentences that cleared that threshold); `build_similarity_heatmap()` (structures per-chunk scores for the UI's bar chart); and `compute_word_saliency()`, which implements the paper's leave-one-out saliency method exactly: remove one word from a sentence, recompute its embedding similarity to the source chunk, and the drop in similarity is that word's saliency. A lexical-overlap fallback (`compute_word_saliency_lexical()`) is used if no embedder is available.

7.5 Detection layer

src/detection/nli_detector.py

`NLIDetector.score_pair()` runs a CrossEncoder over (premise=best-matching chunk, hypothesis=sentence) and returns entailment/neutral/contradiction probabilities, taking entailment as `nli_support_score`. If the DeBERTa model fails to load, `_heuristic_score()` falls back to stop-word-filtered lexical overlap as a crude entailment proxy, so the pipeline degrades rather than crashing.

src/detection/bertscore_detector.py

`BERTScoreDetector.score_sentence()` computes BERTScore precision/recall/F1 between a generated sentence and each retrieved reference chunk using the bert_score library with a distilbert-base-uncased backbone, keeping the best (max F1) match. A cosine-similarity fallback is used if the bert_score package isn't installed.

src/detection/hallucination_engine.py

Defines `SentenceVerdict` (a dataclass holding a sentence's NLI/BERTScore/cosine scores, the aggregated support_score, its classification, and source attribution) and `HallucinationEngine`, whose constructor asserts the three weights sum to 1.0 and whose `compute_support_score()` implements the weighted-sum Trust Score formula from Section 5.4 exactly.

7.6 Controller layer

src/controller/decision_controller.py

Defines the three output states (ANSWER_WITH_CITATIONS, PARTIAL_ANSWER, BLOCKED) and `ControlledResponse`, the final structured object returned to the UI: final text, overall score, per-sentence verdicts, cited sources, hallucination ratio, and the retrieved chunks (so a BLOCKED response can still show the user the raw evidence instead of nothing at all).

7.7 Orchestration & UI

src/pipeline.py

`RAGHallucinationPipeline` reads every section of config.yaml in its `__init__` (paths, ingestion, retrieval, generation, nli, bertscore, detection, controller) and lazy-loads all six models in `_load_models()`. `index_documents()` runs ingestion through FAISS build/save (skipping re-embedding if an index already exists on disk, unless force_reindex=True). `query()` is the single entry point that runs all six pipeline stages in order and returns one result object.

ui/app.py

A Streamlit chat interface (~450 lines). Sidebar: a 'Build Document Index' button, and live sliders for all three detection weights and both decision thresholds, which are pushed directly onto the already-loaded pipeline object so re-running a query with different weights requires no model reload. Main panel: a ChatGPT-style message history where each assistant turn renders a status badge, color-coded sentences (green/yellow/red-strikethrough for Supported/Weakly Supported/Hallucinated) with inline citations, and an expandable 'Explainability & Detection Logs' panel with five summary metrics, a per-claim score table, a radar chart and a horizontal bar chart (Plotly), word-saliency heatmaps, and the raw retrieved chunks.

8. Implementation Status — What's Verified vs. What's Claimed

This is the most important section of this guide to read carefully before Review 2 or before submitting the thesis. It separates three things that are easy to blur together: (a) code that is genuinely implemented and working, (b) numbers that the paper draft reports as experimental results, and (c) numbers that appear in the project's own supporting documents. These three do not currently agree with each other, and a panel or examiner who cross-checks them will notice.

8.1 What is genuinely implemented and verified in code today

Confirmed by direct code review (this is not self-reported — every file in Section 7 was read in full):

- All six pipeline stages are real, working code, not stubs: ingestion & chunking, Sentence-BERT + FAISS retrieval, local LLM generation via Ollama (with an honest, clearly-labeled fallback if Ollama isn't running), the XAI explainability layer (attribution, provenance, word saliency), the three-signal hallucination detector (NLI + BERTScore + cosine), and the tri-state decision controller.
- Every weight and threshold (0.45/0.35/0.20 and 0.75/0.50) is read from config.yaml and used identically in the pipeline code, the Streamlit UI's live sliders, and the paper's formulas — these numbers are consistent everywhere.
- The Streamlit chat UI is a complete, working application with live detection logs, saliency heatmaps, and score breakdowns — verified by reading ui/app.py end to end.
- As of this guide, the real knowledge base contains **21 documents and 94 indexed chunks** (8 original domain/research documents plus 12 new reference notes on RAG, embeddings, FAISS, NLI, BERTScore, XAI, hallucination, chunking, prompt engineering, tri-state control, and evaluation metrics, added to broaden what the system can answer). A browser-based backup of the chat UI, running the same real corpus and the same scoring formula client-side, was built and tested against 12 varied questions — 11 returned a real, cited answer and the one genuinely off-topic control question correctly blocked.

8.2 What the paper draft reports as experimental results — not yet reproduced

The journal paper's Section IV/V describes an experiment at a scale that does not match the current codebase, and the specific numbers reported there have not been regenerated by actually running this code:

- The paper's experimental setup: **~250 domain documents, ~9,800 indexed chunks, a locally hosted Llama 3 8B Instruct model, and a 400-query benchmark (250 answerable + 150 hallucination-inducing)**. The current real corpus is 21 documents / 94 chunks, and the default generator model in config.yaml is Mistral, not Llama 3 8B (both are supported by Ollama, but the larger experiment as described was not actually run).
- The paper's headline results — Precision 0.912, Recall 0.884, **F1 0.898**, **AUC-ROC 0.951**, Recall@5 0.934, BERTScore F1 0.917, Provenance Score 0.936 — along with the full ablation study (Table 4: NLI-only F1 0.856, BERTScore-only 0.792, Cosine-only 0.729, NLI+BERTScore 0.878) and the comparison against a conventional RAG baseline (Table 5: 18.6% vs 2.1% hallucinated sentences) are not reproduced by any evaluation script currently in the repository.
- `evaluation/evaluator.py`'s default code path (when called with no pipeline argument) does not run these models at all — it hard-codes a support score of 0.82 for every claim labeled 'supported' and 0.25 for every claim labeled 'hallucinated', directly from the ground-truth label itself, with a comment reading 'Simulate a reasonable prediction for demo purposes.' The resulting evaluation_report.json (Precision/Recall/F1/AUROC all exactly 1.0) is a demonstration of the scoring arithmetic, not a real measurement. A legitimate code path already exists in the same file (`evaluate_with_pipeline(test_set, pipeline=<real instance>)`) but is not the one invoked by default.

None of this means the framework doesn't work, or that the novelty claim is wrong — the architecture, formulas, and code are sound and genuinely implement what the paper describes. It means the specific performance numbers in the current paper draft are a target experimental design, not a result anyone has actually measured on this codebase yet, and presenting them as achieved results without re-running the real evaluation path would not hold up under scrutiny.

8.3 A second, separate inconsistency — the project's own internal docs disagree with each other

Independent of the paper, the project's own `docs/` folder and PDF-generation scripts contain numbers that conflict with each other, which is worth fixing before they're presented together:

- `docs/01_DATA_INGESTION.md` and `docs/03_EMBEDDING_AND_FAISS.md` describe the real 9-document corpus (~200 chunks), while `docs/09_TOOLS_AND_VERSIONS.md` and `docs/10_PANEL_QA_COMPLETE.md` cite '9,800 chunks' — that figure actually describes a planned MS MARCO merge (10,000 passages) that was never completed, not the real document corpus. The two numbers describe different things but are presented side by side as if comparable.
- `docs/11_PRESENTATION_SCRIPT.md` (written for Phase 1 / Review 1) explicitly instructs: 'Do NOT present results, ablation studies, or comparisons — those belong to Phase 2,' and marks the detection modules and decision controller as 'In Progress.' Meanwhile docs/04, 06, 07, and 10 present the paper's F1/AUC/provenance numbers as already-achieved, finished results. Both cannot be true at once — before Review 2, decide which narrative this project is actually presenting and make the rest of the documentation consistent with it.
- `project_docs/generate_documentation_pdf.py` contains its own hard-coded literature-review table (claiming '70 research papers,' a full year-by-year distribution, and specific file sizes for each corpus document) that isn't computed from anything on disk and doesn't match the corpus this guide verified. `project_docs/generate_pdf.py`'s Project_Plan.pdf has yet a third, independent set of *target* numbers (F1 >= 0.82, a planned 150-pair golden evaluation set) that differ again from both of the above.

Recommendation: before the panel sees more than one of these documents, pick a single honest narrative — for example, "the pipeline and formulas are fully implemented; the large-scale evaluation described in the paper is planned for Semester 3" — and update docs/04, 06, 07, 10, and the documentation-PDF script to match it, rather than letting a panelist discover the discrepancy themselves.

9. Datasets — Current State & Recommended Expansion

9.1 Currently in use

Dataset	Size	Status
Domain document corpus (data/corpus/)	21 documents, 94 indexed chunks (512-word chunks, 50-word overlap)	Fully indexed and live in both the real FAISS index and the
MS MARCO subset (data/corpus/msmarco_2000pass)	2,000 passages (Passage ID N: text format)	Prepared on disk but not yet merged into the FAISS index –

9.2 Recommended additions for Semester 3

These are real, well-known, offline-friendly datasets that would let the project run a genuine evaluation (replacing the fabricated evaluator.py numbers) rather than adding more general trivia:

Dataset	Size	Why it fits
HaluEval (HuggingFace: flowaicom/HaluEval)	10,000 QA examples, ~3.5MB	Closest match to this project's exact problem: correct vs. hallucinated answers with
TruthfulQA (HuggingFace: truthfulqa/truthful_qa)	817 questions, 38 categories, ~500KB	Designed specifically to probe confident-sounding falsehoods — good for stress-tes
RAGTruth (ACL 2024)	18,000 span-level annotated hallucinations	The most directly relevant published benchmark for RAG-specific hallucination, use
FEVER	185,000 claims with Wikipedia evidence	A second domain beyond the current academic-paper corpus, for claim verification

HaluEval is the recommended starting point — it directly fixes the evaluator.py fabrication issue flagged in Section 8.2 with the least integration work.

10. Roadmap for Semester 3

- **Fix the evaluation path.** Point evaluator.py's default run at the real `pipeline=RAGHallucinationPipeline()` path instead of the hard-coded demo branch, and re-run it against HaluEval and/or TruthfulQA to get real Precision/Recall/F1/AUROC numbers.
- **Merge MS MARCO properly.** Decide on a realistic passage count (the prepared 2,000-passage subset, or regenerate a larger one with process_msmarco.py) and actually index it, rather than leaving it prepared but unindexed.
- **Reconcile the internal docs.** Update docs/04, 06, 07, 10, and the PDF generator scripts so every document states the same, currently-true numbers (corpus size, implementation status) rather than three different stories.
- **Run the ablation study for real.** The paper's Table 4 (NLI-only vs. BERTScore-only vs. Cosine-only vs. combined) is a well-designed experiment — it just needs to be executed against the real pipeline and a real test set instead of reported as-is.
- **Decide on the generator model and re-run at that scale.** Either commit to Mistral (the current config default) and update the paper's methodology section to match, or actually pull and test Llama 3 8B as originally described.
- **Domain-specific extension.** The paper's own future-work section suggests healthcare, legal, and enterprise knowledge management as target domains — any of these would make a strong Semester 3 case study once the evaluation pipeline above is real.

End of guide. This document was generated from the actual project folder, the actual journal paper text, and the actual source code — every claim above was checked against one of those three primary sources.